JAVA 8 TO JAVA 21

A code-first, visual, interactive learning textbook

Every release from Java 9 to Java 21 | Why it exists | Before and after | Diagrams | Caveats | Interview reasoning

HOW TO USE THIS BOOK
For each lesson: read the situation; run the old code; predict the new code; trace the diagram; answer the interview question before reading its explanation. Java 21 is the default practice JDK. Preview history is labelled explicitly.

Your learning map

Java 9-11 collections, modules, APIs → Java 12-17 language features → Java 18-21 concurrency and JVM

Plain-English vocabulary

Term	Meaning
Reference	A value that identifies an object; copying it does not copy the object.
Mutable	An object whose state can change after creation.
Unmodifiable list	A list that rejects add, remove, and set through that list.
Snapshot	A separate collection preserving membership at a particular moment.
Shallow copy	A new container whose elements can still point to the same objects.
API	Methods and types other code can call.
JVM	The runtime that executes Java bytecode.
Preview	A feature available for feedback but not finalized in that release.

Version-by-version route

Release	What to study
9	Collection factories: List.of / Set.of / Map.of, Stream takeWhile / dropWhile / ofNullable, Optional additions, Private interface methods, JPMS: modules and module-info.java, JShell and try-with-resources
10	var: local variable type inference, copyOf and unmodifiable collectors
11	String and file convenience APIs, HTTP Client API, Optional.isEmpty / Predicate.not / source launcher
12	Switch expressions: first preview
13	Text blocks: first preview
14	Switch expressions: finalized, Helpful NullPointerExceptions
15	Text blocks: finalized, Hidden classes and JVM background
16	Records, Pattern matching for instanceof, Stream.toList()
17	Sealed classes, Strong encapsulation and migration
18	UTF-8 default charset, Simple Web Server and Javadoc snippets
19	Virtual threads: first preview, Record patterns: first preview
20	Scoped values and structured concurrency: incubation/preview, Virtual thread and record pattern second previews
21	Virtual threads: finalized, Pattern matching for switch: finalized, Record patterns: finalized, Sequenced collections, Generational ZGC and GC reasoning, Java 21 preview features and migration checklist

LESSON 01 | JAVA 9 | Collection factories: List.of / Set.of / Map.of

1 The real problem (read this first)

Your payment service has three allowed methods. Callers should read them, but should not insert a fourth method. First ask: does the caller need a live list or a fixed snapshot?

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
List<String> a = new ArrayList<>();
a.add("UPI"); a.add("CARD");
List<String> fixed = Arrays.asList("UPI", "CARD");
List<String> view = Collections.unmodifiableList(a);
List<String> snapshot = Collections.unmodifiableList(
    new ArrayList<>(a));
```

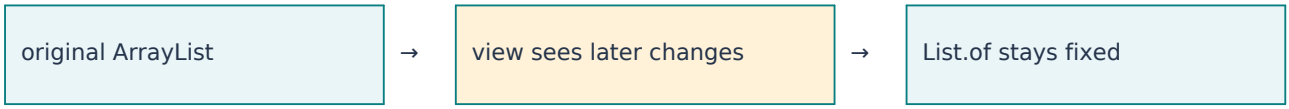
3 What changed and when

A fixed-size list is not the same as an unmodifiable list. Arrays.asList allows set but not add; an unmodifiable view reflects changes to its backing list. List.of makes a new unmodifiable container without retaining a caller-supplied array.

JAVA 9 FEATURE / MODERN PRACTICE

```
List<String> methods = List.of("UPI", "CARD");
Set<String> roles = Set.of("ADMIN", "USER");
Map<String, Integer> limits = Map.of("UPI", 10, "CARD", 5);
// Java 10: List.copyOf(existingList)
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
String[] x = {"UPI", "CARD"};
List<String> backed = Arrays.asList(x);
List<String> independent = List.of(x);
x[0] = "CASH";
System.out.println(backed);      // [CASH, CARD]
System.out.println(independent); // [UPI, CARD]

List<String> original = new ArrayList<>(List.of("A"));
List<String> live = Collections.unmodifiableList(original);
List<String> frozen = List.copyOf(original); // Java 10
original.add("B");
System.out.println(live);   // [A, B]
System.out.println(frozen); // [A]
```

6 Go deeper

ADVANCED EXAMPLE / COMMAND

```
class Payment { String method; Payment(String m){method=m;} }
Payment p = new Payment("UPI");
List<Payment> immutableList = List.of(p);
p.method = "CARD";
System.out.println(immutableList.get(0).method); // CARD
```

7 When to use it - and when not to

CAUTION / TRADE-OFF

List.of rejects null. Set.of rejects duplicates. Map.of rejects duplicate keys. None deep-copy mutable element objects. List.copyOf belongs to Java 10, not Java 9.

8 Interview corner: predict, then explain

QUESTION: What prints after String[] x={"A"}; List<String> l=List.of(x); x[0]="B"; print(l)?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: [A]. List.of copied the element reference into an independent list; changing the array slot does not change that list.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Collection factories: List.of / Set.of / Map.of. Release: Java 9. Core idea: A fixed-size list is not the same as an unmodifiable list. Arrays.asList allows set but not add; an unmodifiable view reflects changes to its backing list. List.of makes a new unmodifiable container without retaining a caller-supplied array.

Watch out: List.of rejects null. Set.of rejects duplicates. Map.of rejects duplicate keys. None deep-copy mutable element objects. List.copyOf belongs to Java 10, not Java 9.

LESSON 02 | JAVA 9 | Stream takeWhile / dropWhile / ofNullable

1 The real problem (read this first)

A sorted temperature log starts with safe values then crosses a threshold. You want the leading safe readings, not every safe reading anywhere in the list.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
List<Integer> a = List.of(2, 4, 9, 3);
List<Integer> first = new ArrayList<>();
for (int n : a) {
    if (n >= 5) break;
    first.add(n);
}
```

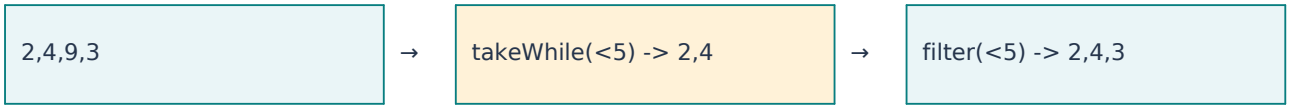
3 What changed and when

takeWhile stops at the first failure. filter checks all elements. dropWhile discards the leading matching prefix then keeps the remainder. ofNullable gives zero or one element.

JAVA 9 FEATURE / MODERN PRACTICE

```
List<Integer> a = List.of(2, 4, 9, 3);
System.out.println(a.stream().takeWhile(n -> n < 5).toList());
System.out.println(a.stream().dropWhile(n -> n < 5).toList());
Stream.ofNullable(possiblyNull).forEach(System.out::println);
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
Stream<Integer> s = Stream.of(2, 4, 9, 3);
System.out.println(s.takeWhile(n -> n < 5)
                    .collect(Collectors.toList()));
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
For unordered streams, takeWhile does not mean a predictable prefix. Stream.toList is Java 16; use collect(toList()) if running on Java 9.

8 Interview corner: predict, then explain

QUESTION: Why is takeWhile(<5) different from filter(<5) on [2,4,9,3]?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: takeWhile returns [2,4]; filter returns [2,4,3]. The first stops when it encounters 9.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Stream takeWhile / dropWhile / ofNullable. Release: Java 9. Core idea: takeWhile stops at the first failure. filter checks all elements. dropWhile discards the leading matching prefix then keeps the remainder. ofNullable gives zero or one element.

Watch out: For unordered streams, takeWhile does not mean a predictable prefix. Stream.toList is Java 16; use collect(toList()) if running on Java 9.

LESSON 03 | JAVA 9 | Optional additions

1 The real problem (read this first)

A lookup may return a user or no user. You need a fallback Optional, a branch for missing values, or to combine present values into a stream.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
if (user.isPresent()) {
    System.out.println(user.get());
} else {
    System.out.println("Missing");
}
```

3 What changed and when

These additions reduce repeated isPresent/get checks. Optional represents possible absence, not a special kind of null-safe mutable box.

JAVA 9 FEATURE / MODERN PRACTICE

```
user.ifPresentOrElse(
    System.out::println,
    () -> System.out.println("Missing"));
Optional<String> fallback = user.or(() -> Optional.of("guest"));
Stream<String> zeroOrOne = user.stream();
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
Optional<String> x = Optional.empty();
System.out.println(x.or(() -> Optional.of("fallback")).get());
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
or() supplier must return a non-null Optional; Optional.orElse evaluates its argument eagerly, whereas orElseGet evaluates lazily.

8 Interview corner: predict, then explain

QUESTION: If Optional.of("A").orElse(expensive()) is used, is expensive() called?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: Yes. Java evaluates method arguments before invoking orElse. Use orElseGet(() -> expensive()) to defer work.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Optional additions. Release: Java 9. Core idea: These additions reduce repeated isPresent/get checks. Optional represents possible absence, not a special kind of null-safe mutable box.

Watch out: or() supplier must return a non-null Optional; Optional.orElse evaluates its argument eagerly, whereas orElseGet evaluates lazily.

LESSON 04 | JAVA 9 | Private interface methods

1 The real problem (read this first)

Two default methods in one interface repeat the same validation logic. You want to share code without exposing that helper as part of the interface API.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
interface Validator {
    default boolean validA(String x) { return x != null && !x.isBlank(); }
    default boolean validB(String x) { return x != null && !x.isBlank(); }
}
```

3 What changed and when

Java 8 interfaces could have default and static methods but not private helper methods. Java 9 lets shared implementation details stay hidden.

JAVA 9 FEATURE / MODERN PRACTICE

```
interface Validator {
    default boolean validA(String x) { return check(x); }
    default boolean validB(String x) { return check(x); }
    private boolean check(String x) {
        return x != null && !x.trim().isEmpty();
    }
}
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
interface Greeting {
    default String hi() { return decorate("Hi"); }
    private String decorate(String s) { return "[" + s + "]"; }
}
```

7 When to use it - and when not to

> CAUTION / TRADE-OFF
> isBlank() was added in Java 11, so a Java 9 example must use trim().isEmpty() or equivalent.

8 Interview corner: predict, then explain

QUESTION: Can an implementing class call interface private helper directly?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. It is only accessible inside that interface.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Private interface methods. Release: Java 9. Core idea: Java 8 interfaces could have default and static methods but not private helper methods. Java 9 lets shared implementation details stay hidden.

Watch out: isBlank() was added in Java 11, so a Java 9 example must use trim().isEmpty() or equivalent.

LESSON 05 | JAVA 9 | JPMS: modules and module-info.java

1 The real problem (read this first)

A large application has library code that other teams should use, plus internal implementation classes they should not depend on. Before modules, public classes on the classpath were broadly visible.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
// Classpath: public class can be used from another JAR.
// Package name alone does not declare module boundaries.
public class PaymentApi { public String name(){return "pay";} }
```

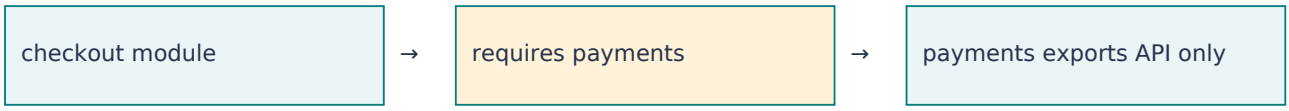
3 What changed and when

A module is a named group of packages. exports exposes selected packages; requires states a dependency. The classpath is the traditional way Java finds classes; the module path adds module-level rules.

JAVA 9 FEATURE / MODERN PRACTICE

```
// module-info.java in payments module
module payments {
    exports com.example.payments.api;
}
// module-info.java in checkout module
module checkout {
    requires payments;
}
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
// payments/src/module-info.java
module payments { exports demo.pay; }
// payments/src/demo/pay/Api.java
package demo.pay;
public class Api { public String ping(){return "ok";} }
// checkout/src/module-info.java
module checkout { requires payments; }
```

6 Go deeper

ADVANCED EXAMPLE / COMMAND

```
javac -d out/payments payments/src/module-info.java     payments/src/demo/pay/Api.java
javac --module-path out -d out/checkout     checkout/src/module-info.java checkout/src/demo/app/Main.java
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
exports is not the same as opens: reflection may need opens. Split packages and automatic modules complicate migration. Most applications do not need to modularize immediately.

8 Interview corner: predict, then explain

QUESTION: Does public class in a non-exported module package become usable by another named module?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. Public is necessary but module exports is also required for ordinary cross-module access.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: JPMS: modules and module-info.java. Release: Java 9. Core idea: A module is a named group of packages. exports exposes selected packages; requires states a dependency. The classpath is the traditional way Java finds classes; the module path adds module-level rules.

Watch out: exports is not the same as opens: reflection may need opens. Split packages and automatic modules complicate migration. Most applications do not need to modularize immediately.

LESSON 06 | JAVA 9 | JShell and try-with-resources

1 The real problem (read this first)

You want to try a single expression without writing a whole class, and you want a resource already declared outside try to be closed automatically.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
// Java 8: write a class + main to test an expression.
try (BufferedReader r = new BufferedReader(reader)) {
    System.out.println(r.readLine());
}
```

3 What changed and when

JShell is an interactive Java REPL: type an expression and see the result. Java 9 also lets try-with-resources use an existing final or effectively final variable.

JAVA 9 FEATURE / MODERN PRACTICE

```
// Terminal: jshell
jshell> 2 + 3
$1 ==> 5
// Java 9: existing effectively-final resource
BufferedReader r = new BufferedReader(reader);
try (r) { System.out.println(r.readLine()); }
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
// JShell commands:
/help
/vars
/exit
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
JShell is for experiments, not a replacement for tests. A resource cannot be reassigned before use in try(r) if it must be effectively final.

8 Interview corner: predict, then explain

QUESTION: Does try(r) close r after the block?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: Yes, even if the block throws, assuming the resource was successfully acquired.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: JShell and try-with-resources. Release: Java 9. Core idea: JShell is an interactive Java REPL: type an expression and see the result. Java 9 also lets try-with-resources use an existing final or effectively final variable.

Watch out: JShell is for experiments, not a replacement for tests. A resource cannot be reassigned before use in try(r) if it must be effectively final.

LESSON 07 | JAVA 10 | var: local variable type inference

1 The real problem (read this first)

A variable has an obvious type on the right side; repeating the long generic type makes the line harder to scan.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
Map<String, List<Integer>> scores = new HashMap<>();
for (String name : scores.keySet()) { System.out.println(name); }
```

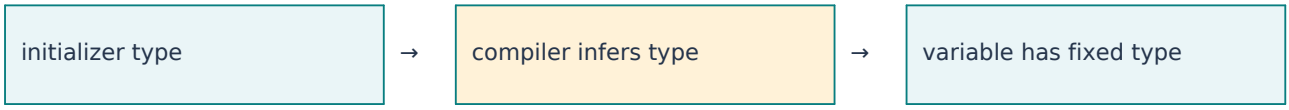
3 What changed and when

var asks the compiler to infer the local variable type from the initializer. Java remains statically typed: var does not mean dynamic typing.

JAVA 10 FEATURE / MODERN PRACTICE

```
var scores = new HashMap<String, List<Integer>>();
for (var name : scores.keySet()) { System.out.println(name); }
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
var count = 10;         // int
var names = List.of("A"); // List<String>
// var unknown = null;  // compilation error
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Not allowed for fields, method parameters, or uninitialized locals. var x = null fails because there is no type to infer. Avoid var when it hides meaning.

8 Interview corner: predict, then explain

QUESTION: Does var x = "Java"; x = 42; compile?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. x is inferred as String; assigning int is invalid.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: var: local variable type inference. Release: Java 10. Core idea: var asks the compiler to infer the local variable type from the initializer. Java remains statically typed: var does not mean dynamic typing.

Watch out: Not allowed for fields, method parameters, or uninitialized locals. var x = null fails because there is no type to infer. Avoid var when it hides meaning.

LESSON 08 | JAVA 10 | copyOf and unmodifiable collectors

1 The real problem (read this first)

You receive a mutable list from another component and need a stable read-only snapshot of its current membership.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
List<String> stable = Collections.unmodifiableList(
    new ArrayList<>(source));
```

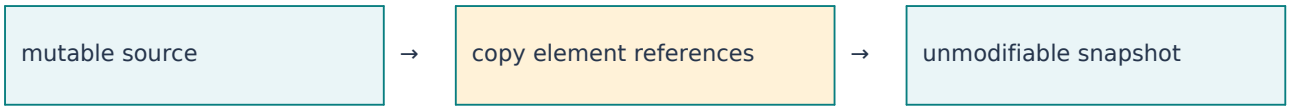
3 What changed and when

copyOf simplifies the defensive-copy pattern. Defensive copy means making your own container so later changes to the source cannot change your container.

JAVA 10 FEATURE / MODERN PRACTICE

```
List<String> stable = List.copyOf(source);
Set<String> stableSet = Set.copyOf(source);
Map<String, Integer> stableMap = Map.copyOf(map);
var result = stream.collect(Collectors.toUnmodifiableList());
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
List<String> source = new ArrayList<>(List.of("A"));
List<String> snapshot = List.copyOf(source);
source.add("B");
System.out.println(snapshot); // [A]
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
No null elements or keys/values. Copy is shallow. Set.copyOf may collapse duplicate elements rather than throwing; Set.of rejects duplicates.

8 Interview corner: predict, then explain

QUESTION: If source.add("B") happens after List.copyOf(source), does snapshot gain B?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. Its element references were fixed at creation time.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: copyOf and unmodifiable collectors. Release: Java 10. Core idea: copyOf simplifies the defensive-copy pattern. Defensive copy means making your own container so later changes to the source cannot change your container.

Watch out: No null elements or keys/values. Copy is shallow. Set.copyOf may collapse duplicate elements rather than throwing; Set.of rejects duplicates.

LESSON 09 | JAVA 11 | String and file convenience APIs

1 The real problem (read this first)

You frequently check blank input, strip surrounding whitespace, split lines, or read a small text file. The old code works but repeats low-level steps.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
boolean blank = s.trim().isEmpty();
String text = new String(Files.readAllBytes(path),
                         StandardCharsets.UTF_8);
```

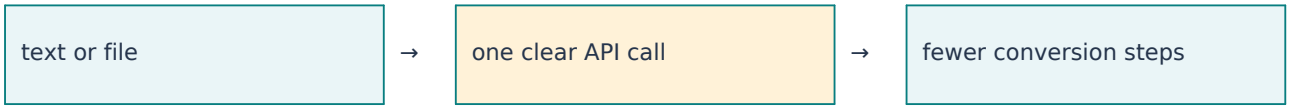
3 What changed and when

isBlank understands Unicode whitespace more broadly than trim().isEmpty(). readString/writeString use UTF-8 by default and simplify small text-file tasks.

JAVA 11 FEATURE / MODERN PRACTICE

```
boolean blank = s.isBlank();
String clean = s.strip();
String repeated = "-".repeat(3);
long lines = s.lines().count();
String text = Files.readString(path);
Files.writeString(path, "hello");
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
System.out.println("  x  ".strip());
System.out.println("ha".repeat(3)); // hahaha
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
readString loads the whole file into memory; use streaming for huge files. strip and trim have different whitespace rules.

8 Interview corner: predict, then explain

QUESTION: Does " ".isBlank() return true?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: Yes. It consists only of whitespace characters.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: String and file convenience APIs. Release: Java 11. Core idea: isBlank understands Unicode whitespace more broadly than trim().isEmpty(). readString/writeString use UTF-8 by default and simplify small text-file tasks.

Watch out: readString loads the whole file into memory; use streaming for huge files. strip and trim have different whitespace rules.

LESSON 10 | JAVA 11 | HTTP Client API

1 The real problem (read this first)

A service calls a remote API. You want a standard HTTP client with clearer request/response objects and async support.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
HttpURLConnection c = (HttpURLConnection)
    new URL("https://example.com").openConnection();
c.setRequestMethod("GET");
```

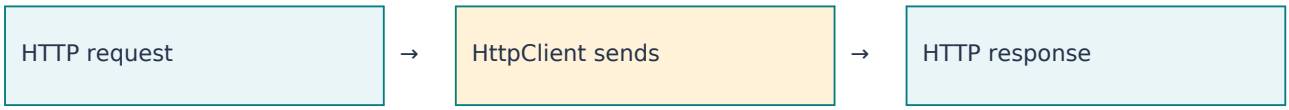
3 What changed and when

The Java 11 standard client supports HTTP/2 and asynchronous calls. A request describes what to send; a response contains status, headers, and body.

JAVA 11 FEATURE / MODERN PRACTICE

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder(
    URI.create("https://example.com")).GET().build();
HttpResponse<String> res = client.send(req,
    HttpResponse.BodyHandlers.ofString());
System.out.println(res.statusCode());
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
client.sendAsync(req, HttpResponse.BodyHandlers.ofString())
      .thenApply(HttpResponse::body)
      .thenAccept(System.out::println);
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Network calls fail, time out, or return errors. Check status codes and configure timeouts. send is blocking; sendAsync returns CompletableFuture.

8 Interview corner: predict, then explain

QUESTION: Does status 404 automatically mean send() throws IOException?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. HTTP error status is still a valid HTTP response; inspect statusCode().

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: HTTP Client API. Release: Java 11. Core idea: The Java 11 standard client supports HTTP/2 and asynchronous calls. A request describes what to send; a response contains status, headers, and body.

Watch out: Network calls fail, time out, or return errors. Check status codes and configure timeouts. send is blocking; sendAsync returns CompletableFuture.

LESSON 11 | JAVA 11 | Optional.isEmpty / Predicate.not / source launcher

1 The real problem (read this first)

You want readable checks for missing values, readable filtering of nonblank text, and to run a tiny Java program without a separate javac command.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
if (!result.isPresent()) { System.out.println("missing"); }
items.stream().filter(x -> !x.isBlank());
```

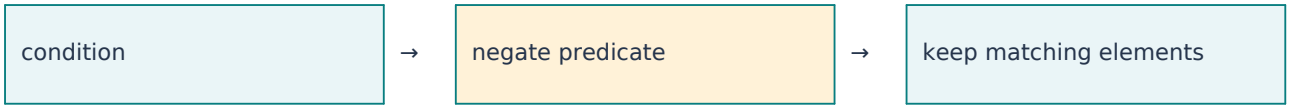
3 What changed and when

isEmpty is the opposite of isPresent. Predicate.not wraps a condition in logical NOT. Source launcher compiles a single source file in memory and runs it.

JAVA 11 FEATURE / MODERN PRACTICE

```
if (result.isEmpty()) { System.out.println("missing"); }
items.stream().filter(Predicate.not(String::isBlank));
// Terminal: java Hello.java
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
List<String> words = List.of("hi", " ", "bye");
System.out.println(words.stream()
    .filter(Predicate.not(String::isBlank)).toList());
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Predicate.not needs a method reference or Predicate; readability is subjective. Single-file launching is not a build system.

8 Interview corner: predict, then explain

QUESTION: What does Predicate.not(String::isBlank) keep?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: Strings for which isBlank() is false.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Optional.isEmpty / Predicate.not / source launcher. Release: Java 11. Core idea: isEmpty is the opposite of isPresent. Predicate.not wraps a condition in logical NOT. Source launcher compiles a single source file in memory and runs it.

Watch out: Predicate.not needs a method reference or Predicate; readability is subjective. Single-file launching is not a build system.

LESSON 12 | JAVA 12 | Switch expressions: first preview

1 The real problem (read this first)

A switch chooses a value such as a label. In older Java, you declare a variable, assign it in branches, and remember break statements.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
String label;
switch (day) {
    case 1: label = "Mon"; break;
    case 2: label = "Tue"; break;
    default: label = "Other";
}
```

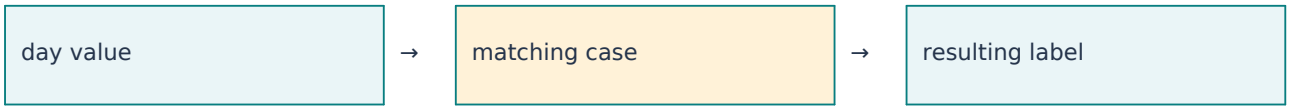
3 What changed and when

The new form returns a value directly and arrow cases avoid accidental fall-through. Preview means the language design was available for testing but not yet finalized.

JAVA 12 FEATURE / MODERN PRACTICE

```
// Java 12 preview; finalized Java 14
String label = switch (day) {
    case 1 -> "Mon";
    case 2 -> "Tue";
    default -> "Other";
};
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
int n = 2;
String x = switch (n) {
    case 1 -> "one";
    default -> "other";
};
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
In Java 12 this required --enable-preview and matching release flags. On Java 21 the finalized syntax works normally.

8 Interview corner: predict, then explain

QUESTION: Why does the new switch not need break for arrow cases?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: Arrow cases do not fall through into the next case.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Switch expressions: first preview. Release: Java 12. Core idea: The new form returns a value directly and arrow cases avoid accidental fall-through. Preview means the language design was available for testing but not yet finalized.

Watch out: In Java 12 this required --enable-preview and matching release flags. On Java 21 the finalized syntax works normally.

LESSON 13 | JAVA 13 | Text blocks: first preview

1 The real problem (read this first)

You store a multiline SQL query or JSON example. Many escaped quotes and newline markers make it hard to read and review.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
String sql = "SELECT id, name\n" +
             "FROM users\n" +
             "WHERE active = true";
```

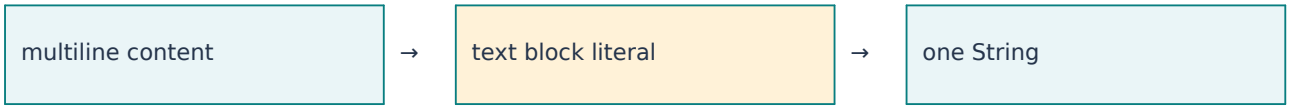
3 What changed and when

A text block is a multiline string literal. It makes the text resemble its intended shape rather than a chain of concatenated strings.

JAVA 13 FEATURE / MODERN PRACTICE

```
// Java 13 preview; finalized Java 15
String sql = """
    SELECT id, name
    FROM users
    WHERE active = true
    """;
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
String json = """
    {"name":"Ada","active":true}
    """;
```

7 When to use it - and when not to

CAUTION / TRADE-OFF

Indentation and trailing newline rules matter. A text block is still a String; it is not SQL injection protection.

8 Interview corner: predict, then explain

QUESTION: Is a text block a different runtime type from String?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. It is a String.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Text blocks: first preview. Release: Java 13. Core idea: A text block is a multiline string literal. It makes the text resemble its intended shape rather than a chain of concatenated strings.

Watch out: Indentation and trailing newline rules matter. A text block is still a String; it is not SQL injection protection.

LESSON 14 | JAVA 14 | Switch expressions: finalized

1 The real problem (read this first)

You need a stable, production-ready way to compute a value with switch. Java 14 finalized the preview from Java 12/13.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
int score;
switch (grade) {
    case "A": score = 5; break;
    default: score = 0;
}
```

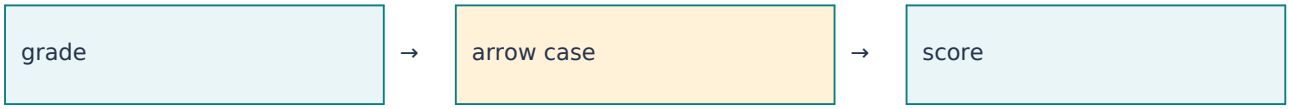
3 What changed and when

Java 14 made switch expressions a permanent language feature. You can also use yield inside a block when a case needs multiple statements.

JAVA 14 FEATURE / MODERN PRACTICE

```
int score = switch (grade) {
    case "A" -> 5;
    case "B" -> 4;
    default -> 0;
};
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
int score = switch (grade) {
    case "A" -> { System.out.println("excellent"); yield 5; }
    default -> 0;
};
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
An expression must cover all possible inputs. A default is often needed; an enum switch may be exhaustive without one.

8 Interview corner: predict, then explain

QUESTION: Why must switch expression cover every possible input?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: Because it must produce a value on every execution path.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Switch expressions: finalized. Release: Java 14. Core idea: Java 14 made switch expressions a permanent language feature. You can also use yield inside a block when a case needs multiple statements.

Watch out: An expression must cover all possible inputs. A default is often needed; an enum switch may be exhaustive without one.

LESSON 15 | JAVA 14 | Helpful NullPointerExceptions

1 The real problem (read this first)

A long chain like order.getCustomer().getAddress().getCity() throws NPE. You need to know which part was null.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
// Old stack trace may only point to the line:
String city = order.getCustomer().getAddress().getCity();
```

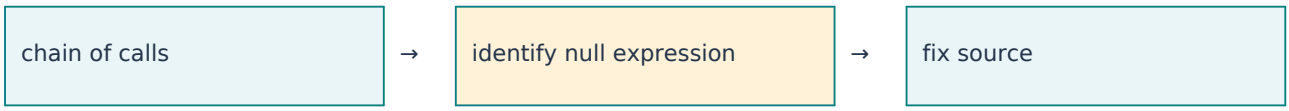
3 What changed and when

The JVM can provide more specific diagnostic messages for many null dereferences. This improves debugging but does not prevent null errors.

JAVA 14 FEATURE / MODERN PRACTICE

```
// Java 14 helpful NPE messages identify the null part:
String city = order.getCustomer().getAddress().getCity();
// Example: cannot invoke getCity() because getAddress() is null
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
Objects.requireNonNull(order, "order required");
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Exact wording varies; do not parse exception text as an API contract. Helpful NPEs do not replace validation.

8 Interview corner: predict, then explain

QUESTION: Do helpful NPE messages automatically make code null-safe?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. They only make failures easier to diagnose.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Helpful NullPointerExceptions. Release: Java 14. Core idea: The JVM can provide more specific diagnostic messages for many null dereferences. This improves debugging but does not prevent null errors.

Watch out: Exact wording varies; do not parse exception text as an API contract. Helpful NPEs do not replace validation.

LESSON 16 | JAVA 15 | Text blocks: finalized

1 The real problem (read this first)

Your SQL, JSON, and HTML literals are difficult to read because of concatenation and escaping. Java 15 finalized text blocks.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
String html = "<h1>Hi</h1>\n" +
              "<p>Welcome</p>";
```

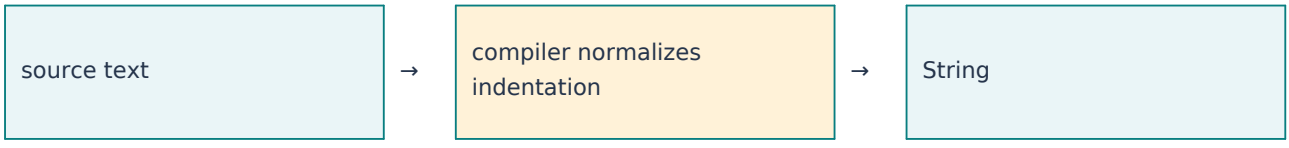
3 What changed and when

Text blocks improve readability without introducing a new String type.

JAVA 15 FEATURE / MODERN PRACTICE

```
String html = """
    <h1>Hi</h1>
    <p>Welcome</p>
    """;
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
System.out.println("""
    Hello
    World
    """.lines().count());
```

7 When to use it - and when not to

> CAUTION / TRADE-OFF
> Still escape three consecutive double quotes when needed. Avoid embedding untrusted values directly into SQL.

8 Interview corner: predict, then explain

QUESTION: Can you call .length() on a text block?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: Yes. It is an ordinary String.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Text blocks: finalized. Release: Java 15. Core idea: Text blocks improve readability without introducing a new String type.

Watch out: Still escape three consecutive double quotes when needed. Avoid embedding untrusted values directly into SQL.

LESSON 17 | JAVA 15 | Hidden classes and JVM background

1 The real problem (read this first)

Frameworks generate helper classes at runtime. Ordinary named classes can be awkward to manage and may leak implementation details.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
// Frameworks previously generated ordinary named classes
// or relied on classloader tricks.
```

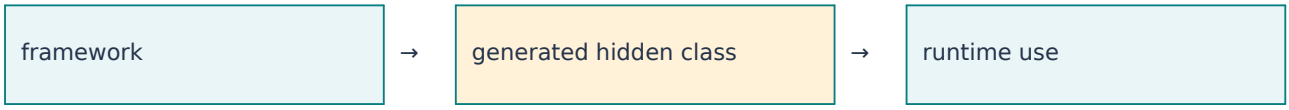
3 What changed and when

Hidden classes are classes that cannot be discovered by their binary name in the normal way. They help runtime frameworks generate implementation helpers.

JAVA 15 FEATURE / MODERN PRACTICE

```
// Java 15: Lookup.defineHiddenClass(...) JVM API
// Mainly used by frameworks, not everyday business code.
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
// Ordinary application code remains ordinary Java:
class PaymentService { void pay() {} }
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
This is an advanced JVM API. Learn the purpose before implementation; most application developers never call it directly.

8 Interview corner: predict, then explain

QUESTION: Should a payment service normally use defineHiddenClass?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. Use normal classes unless building runtime language/framework infrastructure.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Hidden classes and JVM background. Release: Java 15. Core idea: Hidden classes are classes that cannot be discovered by their binary name in the normal way. They help runtime frameworks generate implementation helpers.

Watch out: This is an advanced JVM API. Learn the purpose before implementation; most application developers never call it directly.

LESSON 18 | JAVA 16 | Records

1 The real problem (read this first)

A data carrier holds a name and age. You keep writing constructor, getters, equals, hashCode, and toString for the same fields.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
final class User {
    private final String name;
    private final int age;
    User(String name, int age) { this.name=name; this.age=age; }
    String getName() { return name; }
    int getAge() { return age; }
    // plus equals/hashCode/toString
}
```

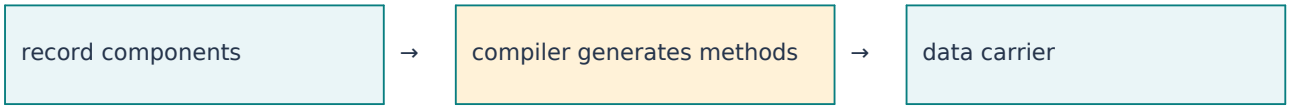
3 What changed and when

A record is a compact class designed to represent data. Java generates a constructor, accessors, equals, hashCode, and toString from its components.

JAVA 16 FEATURE / MODERN PRACTICE

```
record User(String name, int age) {}
User u = new User("Ada", 30);
System.out.println(u.name());
System.out.println(u);
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
record User(String name, int age) {
    User {
        if (age < 0) throw new IllegalArgumentException("age");
    }
}
```

6 Go deeper

ADVANCED EXAMPLE / COMMAND

```
record Team(List<String> members) {
    Team { members = List.copyOf(members); }
}
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Record fields are final, but referenced objects can be mutable. Records cannot extend another class. Validate inputs in a compact constructor.

8 Interview corner: predict, then explain

QUESTION: Is record User(List<String> names) deeply immutable?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. The reference is final, but the list may still be mutable unless defensively copied.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Records. Release: Java 16. Core idea: A record is a compact class designed to represent data. Java generates a constructor, accessors, equals, hashCode, and toString from its components.

Watch out: Record fields are final, but referenced objects can be mutable. Records cannot extend another class. Validate inputs in a compact constructor.

LESSON 19 | JAVA 16 | Pattern matching for instanceof

1 The real problem (read this first)

You check whether an Object is a String, then cast it to String to use String methods. The cast repeats information already checked.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
if (value instanceof String) {
    String s = (String) value;
    System.out.println(s.length());
}
```

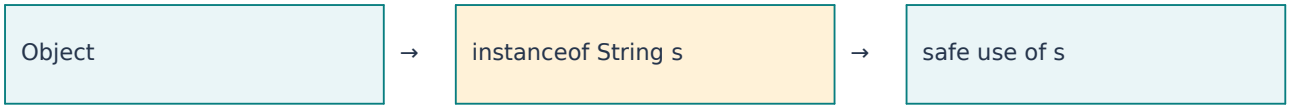
3 What changed and when

The pattern both tests the type and creates a correctly typed variable when the test succeeds.

JAVA 16 FEATURE / MODERN PRACTICE

```
if (value instanceof String s) {
    System.out.println(s.length());
}
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
Object x = "Java";
if (x instanceof String s && s.length() > 2) {
    System.out.println(s.toUpperCase());
}
```

7 When to use it - and when not to

CAUTION / TRADE-OFF

Pattern variable exists only where the compiler knows the test succeeded. instanceof null is false.

8 Interview corner: predict, then explain

QUESTION: Can s be used after the if block?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: Usually no. Its scope is controlled by where the pattern is definitely matched.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Pattern matching for instanceof. Release: Java 16. Core idea: The pattern both tests the type and creates a correctly typed variable when the test succeeds.

Watch out: Pattern variable exists only where the compiler knows the test succeeded. instanceof null is false.

LESSON 20 | JAVA 16 | Stream.toList()

1 The real problem (read this first)

You transform a stream into a list. Java 8 used a collector with more typing.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
List<String> result = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

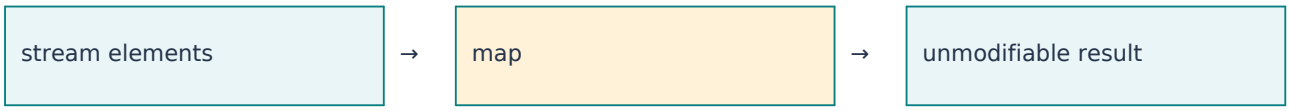
3 What changed and when

The shorter terminal operation directly collects the stream results into an unmodifiable list. Terminal means it finishes stream processing and produces a result.

JAVA 16 FEATURE / MODERN PRACTICE

```
List<String> result = names.stream()
    .map(String::toUpperCase)
    .toList();
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
List<String> result = Stream.of("a", "b").toList();
// result.add("c"); // UnsupportedOperationException
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Stream.toList() returns an unmodifiable list and permits nulls. Collectors.toList() does not promise mutability, even if common implementations return ArrayList.

8 Interview corner: predict, then explain

QUESTION: Can result.add("X") work after Stream.toList()?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. It throws UnsupportedOperationException.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Stream.toList(). Release: Java 16. Core idea: The shorter terminal operation directly collects the stream results into an unmodifiable list. Terminal means it finishes stream processing and produces a result.

Watch out: Stream.toList() returns an unmodifiable list and permits nulls. Collectors.toList() does not promise mutability, even if common implementations return ArrayList.

LESSON 21 | JAVA 17 | Sealed classes

1 The real problem (read this first)

A payment result should be one of Success or Failure. Allowing anyone to add arbitrary subclasses makes handling every possibility harder.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
interface Result {}
class Success implements Result {}
class Failure implements Result {}
// Anyone can add a new implementation.
```

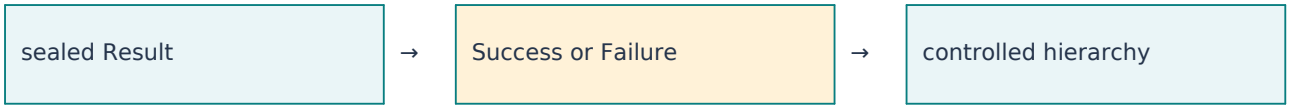
3 What changed and when

sealed limits which classes can extend or implement a type. The permitted types must be final, sealed, or non-sealed as appropriate.

JAVA 17 FEATURE / MODERN PRACTICE

```
sealed interface Result permits Success, Failure {}
record Success(String id) implements Result {}
record Failure(String reason) implements Result {}
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
sealed interface Shape permits Circle, Square {}
record Circle(int radius) implements Shape {}
record Square(int side) implements Shape {}
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Sealed is not the same as final. A non-sealed permitted subclass can open its branch for extension.

8 Interview corner: predict, then explain

QUESTION: Can class Unknown implements Result compile here?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. It is not in the permits list.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Sealed classes. Release: Java 17. Core idea: sealed limits which classes can extend or implement a type. The permitted types must be final, sealed, or non-sealed as appropriate.

Watch out: Sealed is not the same as final. A non-sealed permitted subclass can open its branch for extension.

LESSON 22 | JAVA 17 | Strong encapsulation and migration

1 The real problem (read this first)

A Java 8 library uses internal JDK classes through reflection. After upgrading, the code may fail because those internals are no longer freely accessible.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
// Fragile Java 8-era practice:
// importing sun.* internal APIs or using setAccessible
// on private JDK internals.
```

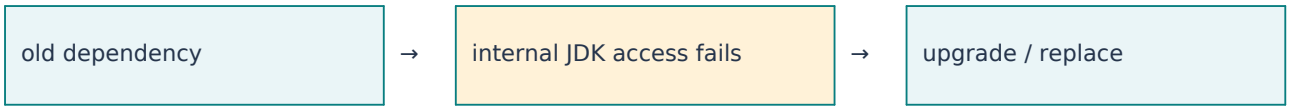
3 What changed and when

Encapsulation means hiding implementation details behind supported APIs. Java 17 strongly restricts many JDK internal packages.

JAVA 17 FEATURE / MODERN PRACTICE

```
// Prefer supported public APIs.
// Diagnose with jdeps and upgrade libraries.
// Temporary --add-opens may help migration,
// but should not be the long-term design.
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
// Terminal diagnostics:
jdeps --jdk-internals your-app.jar
java -version
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Not every Java 8 JAR works unchanged on 17 or 21. Removed Java EE modules (Java 11) may need external dependencies.

8 Interview corner: predict, then explain

QUESTION: Does --add-opens permanently fix an unsupported internal API?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. It can be a temporary workaround, not a guarantee of future compatibility.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Strong encapsulation and migration. Release: Java 17. Core idea: Encapsulation means hiding implementation details behind supported APIs. Java 17 strongly restricts many JDK internal packages.

Watch out: Not every Java 8 JAR works unchanged on 17 or 21. Removed Java EE modules (Java 11) may need external dependencies.

LESSON 23 | JAVA 18 | UTF-8 default charset

1 The real problem (read this first)

The same text file behaves differently on developer laptops and servers because their operating systems choose different default character encodings.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
// Older Java default charset depends on environment:
byte[] bytes = "cafe".getBytes();
```

3 What changed and when

A character encoding maps text characters to bytes. UTF-8 is a widely used encoding. A consistent default reduces machine-specific surprises.

JAVA 18 FEATURE / MODERN PRACTICE

```
// Java 18 standardizes default charset to UTF-8:
byte[] bytes = "cafe".getBytes();
// Explicit is still clearest at boundaries:
byte[] safe = "cafe".getBytes(StandardCharsets.UTF_8);
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
String text = Files.readString(path, StandardCharsets.UTF_8);
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
When reading legacy files written in another encoding, specify that encoding explicitly. The default change does not convert existing files.

8 Interview corner: predict, then explain

QUESTION: Does Java 18 automatically rewrite old ISO-8859-1 files as UTF-8?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. It changes the default used by certain APIs, not stored bytes.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: UTF-8 default charset. Release: Java 18. Core idea: A character encoding maps text characters to bytes. UTF-8 is a widely used encoding. A consistent default reduces machine-specific surprises.

Watch out: When reading legacy files written in another encoding, specify that encoding explicitly. The default change does not convert existing files.

LESSON 24 | JAVA 18 | Simple Web Server and Javadoc snippets

1 The real problem (read this first)

You need to serve static files locally for testing and include runnable-looking code in documentation.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
// Older setup: install separate web server tool.
// Javadoc examples often used preformatted HTML.
```

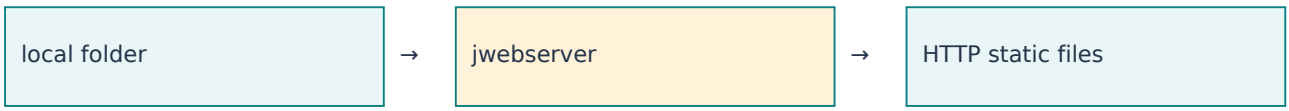
3 What changed and when

The built-in simple web server is useful for local testing, not production hosting. Snippet support makes API examples easier to maintain.

JAVA 18 FEATURE / MODERN PRACTICE

```
// Java 18 terminal:
jwebserver -p 8000
// Javadoc: @snippet tag for code examples.
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
// Terminal in a safe test directory:
jwebserver -p 8000 -d public
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Do not expose private files or treat jwebserver as a hardened production web server.

8 Interview corner: predict, then explain

QUESTION: Should you run a production payment API on jwebserver?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. It is a simple development/testing server.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Simple Web Server and Javadoc snippets. Release: Java 18. Core idea: The built-in simple web server is useful for local testing, not production hosting. Snippet support makes API examples easier to maintain.

Watch out: Do not expose private files or treat jwebserver as a hardened production web server.

LESSON 25 | JAVA 19 | Virtual threads: first preview

1 The real problem (read this first)

A web service waits for databases and network responses. Creating a large number of platform threads can be costly, but writing callback chains is harder to follow.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
ExecutorService pool = Executors.newFixedThreadPool(100);
pool.submit(() -> callRemoteService());
```

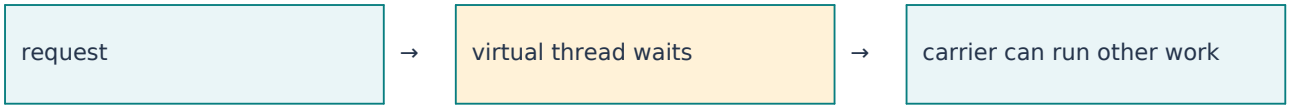
3 What changed and when

Virtual threads are lightweight Java threads scheduled onto a smaller set of operating-system threads. They help scale many tasks that spend time waiting.

JAVA 19 FEATURE / MODERN PRACTICE

```
// Java 19 preview; finalized in Java 21
Thread.startVirtualThread(() -> callRemoteService());
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
// Java 21 finalized API:
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    executor.submit(() -> "done");
}
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Not a speed boost for CPU-bound loops. In Java 19 this required preview flags; use finalized Java 21 APIs for learning.

8 Interview corner: predict, then explain

QUESTION: Do 10,000 virtual threads mean 10,000 OS threads?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. Many virtual threads are scheduled over fewer carrier platform threads.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Virtual threads: first preview. Release: Java 19. Core idea: Virtual threads are lightweight Java threads scheduled onto a smaller set of operating-system threads. They help scale many tasks that spend time waiting.

Watch out: Not a speed boost for CPU-bound loops. In Java 19 this required preview flags; use finalized Java 21 APIs for learning.

LESSON 26 | JAVA 19 | Record patterns: first preview

1 The real problem (read this first)

You have a record Point(x,y). You want to check its type and immediately extract its components without writing several accessor calls.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
if (obj instanceof Point) {
    Point p = (Point) obj;
    int x = p.x(); int y = p.y();
}
```

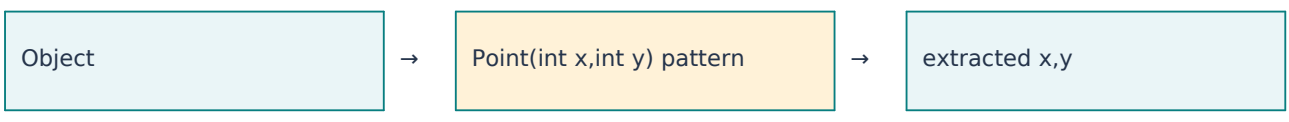
3 What changed and when

A record pattern checks a record shape and binds its component values to variables.

JAVA 19 FEATURE / MODERN PRACTICE

```
// Java 19 preview; finalized Java 21
if (obj instanceof Point(int x, int y)) {
    System.out.println(x + y);
}
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
record Point(int x, int y) {}
Object obj = new Point(2, 3);
if (obj instanceof Point(int x, int y))
    System.out.println(x + y); // 5
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Record patterns work with records, not arbitrary classes. Preview syntax evolved; use Java 21 final form.

8 Interview corner: predict, then explain

QUESTION: Does a record pattern construct a new Point?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. It inspects and extracts values from an existing record.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Record patterns: first preview. Release: Java 19. Core idea: A record pattern checks a record shape and binds its component values to variables.

Watch out: Record patterns work with records, not arbitrary classes. Preview syntax evolved; use Java 21 final form.

LESSON 27 | JAVA 20 | Scoped values and structured concurrency: incubation/preview

1 The real problem (read this first)

A parent task starts subtasks and needs a way to manage their lifetime or share request context without uncontrolled global mutable state.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
// Older approach: ThreadLocal for context;
// CompletableFuture chains or manual futures for subtasks.
```

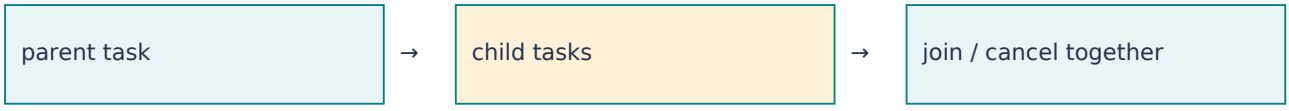
3 What changed and when

Structured concurrency treats related subtasks as one unit so cancellation and failure handling can be coordinated. Scoped values provide a controlled way to share context down a call tree.

JAVA 20 FEATURE / MODERN PRACTICE

```
// Java 20: incubating structured concurrency;
// scoped values were introduced as an incubating API.
// Java 21: both were preview APIs (not final).
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
// Conceptual pseudocode (NOT compilable Java):
// scope.start(fetchUser)
// scope.start(fetchOrders)
// scope.join()
// combine results or cancel on failure
```

7 When to use it - and when not to

CAUTION / TRADE-OFF

These APIs were not finalized in Java 21. Their APIs changed in later releases. Do not teach preview signatures as stable production contracts.

8 Interview corner: predict, then explain

QUESTION: Are structured concurrency and scoped values final Java 21 features?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. Both are preview features in Java 21.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Scoped values and structured concurrency: incubation/preview. Release: Java 20. Core idea: Structured concurrency treats related subtasks as one unit so cancellation and failure handling can be coordinated. Scoped values provide a controlled way to share context down a call tree.

Watch out: These APIs were not finalized in Java 21. Their APIs changed in later releases. Do not teach preview signatures as stable production contracts.

LESSON 28 | JAVA 20 | Virtual thread and record pattern second previews

1 The real problem (read this first)

You are testing preview designs before they become permanent in Java 21. Why should an application developer care about the preview label?

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
// Java 8: platform thread pools + casts/accessors.
```

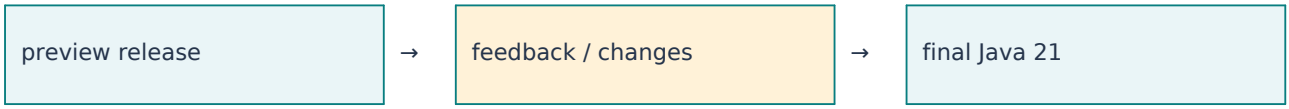
3 What changed and when

A preview feature lets developers try language or platform designs and give feedback before they become stable. Java 20 refined these two features.

JAVA 20 FEATURE / MODERN PRACTICE

```
// Java 20 preview iterations:
// virtual threads (second preview)
// record patterns (second preview)
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
// Java 21 final:
record Point(int x, int y) {}
Object p = new Point(1, 2);
if (p instanceof Point(int x, int y))
    System.out.println(x + y);
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Avoid depending on a preview API in a long-lived library without accounting for possible syntax/API changes.

8 Interview corner: predict, then explain

QUESTION: Should you memorize the Java 20 preview syntax when targeting Java 21?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. Understand its history; learn the finalized Java 21 version.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Virtual thread and record pattern second previews. Release: Java 20. Core idea: A preview feature lets developers try language or platform designs and give feedback before they become stable. Java 20 refined these two features.

Watch out: Avoid depending on a preview API in a long-lived library without accounting for possible syntax/API changes.

LESSON 29 | JAVA 21 | Virtual threads: finalized

1 The real problem (read this first)

Your server handles many requests, each spending most time waiting on network or database I/O. You want straightforward blocking code without a huge fixed pool of platform threads.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
ExecutorService pool = Executors.newFixedThreadPool(200);
Future<String> f = pool.submit(() -> fetch());
System.out.println(f.get());
pool.shutdown();
```

3 What changed and when

A platform thread maps more directly to an OS thread. A virtual thread is a lightweight Java thread; when it blocks in supported operations, its carrier thread can do other work.

JAVA 21 FEATURE / MODERN PRACTICE

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    Future<String> f = executor.submit(() -> fetch());
    System.out.println(f.get());
}
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
Semaphore limit = new Semaphore(20);
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 100; i++) executor.submit(() -> {
        limit.acquire();
        try { return fetch(); }
        finally { limit.release(); }
    });
}
```

6 Go deeper

ADVANCED EXAMPLE / COMMAND

```
// CPU-bound task: virtual threads do not add CPU cores.
// Blocking task: virtual threads often improve scalability.
// Measure throughput, latency, memory, and downstream limits.
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Do not pool virtual threads; limit access to scarce resources with semaphores or connection pools. CPU-heavy work is not automatically faster. Pinning can reduce scalability in some cases.

8 Interview corner: predict, then explain

QUESTION: If database permits 20 connections, do 10,000 virtual threads create 10,000 connections safely?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. The database connection pool still limits active connections; virtual threads do not remove downstream capacity limits.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Virtual threads: finalized. Release: Java 21. Core idea: A platform thread maps more directly to an OS thread. A virtual thread is a lightweight Java thread; when it blocks in supported operations, its carrier thread can do other work.

Watch out: Do not pool virtual threads; limit access to scarce resources with semaphores or connection pools. CPU-heavy work is not automatically faster. Pinning can reduce scalability in some cases.

LESSON 30 | JAVA 21 | Pattern matching for switch: finalized

1 The real problem (read this first)

You receive different message types and must branch by type. Old code uses repeated instanceof checks and casts.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
if (obj instanceof String) {
    String s = (String) obj;
    System.out.println(s.length());
} else if (obj instanceof Integer) {
    Integer n = (Integer) obj;
    System.out.println(n + 1);
}
```

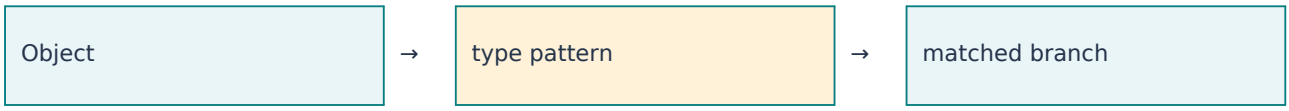
3 What changed and when

Type patterns let switch select a branch based on the runtime type. Java 21 finalized pattern matching for switch, including support for case null.

JAVA 21 FEATURE / MODERN PRACTICE

```
String message = switch (obj) {
    case String s -> "length=" + s.length();
    case Integer n -> "next=" + (n + 1);
    case null -> "missing";
    default -> "unknown";
};
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
Object x = "hello";
System.out.println(switch (x) {
    case String s -> s.toUpperCase();
    default -> "other";
});
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Ordering matters: a broad type pattern can dominate narrower cases and cause compilation errors. A switch expression must be exhaustive.

8 Interview corner: predict, then explain

QUESTION: Can case Object o appear before case String s?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No, if it makes String case unreachable; the compiler rejects dominated cases.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Pattern matching for switch: finalized. Release: Java 21. Core idea: Type patterns let switch select a branch based on the runtime type. Java 21 finalized pattern matching for switch, including support for case null.

Watch out: Ordering matters: a broad type pattern can dominate narrower cases and cause compilation errors. A switch expression must be exhaustive.

LESSON 31 | JAVA 21 | Record patterns: finalized

1 The real problem (read this first)

Your event contains nested records. You want to check the structure and extract fields in one readable step.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
if (obj instanceof Point) {
    Point p = (Point) obj;
    System.out.println(p.x() + p.y());
}
```

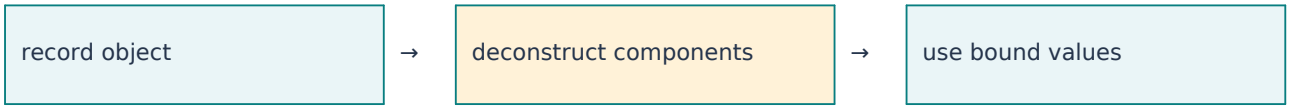
3 What changed and when

Record patterns combine type checking with component extraction, and can be nested to match records inside records.

JAVA 21 FEATURE / MODERN PRACTICE

```
record Point(int x, int y) {}
Object obj = new Point(2, 5);
if (obj instanceof Point(int x, int y)) {
    System.out.println(x + y); // 7
}
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
record Address(String city) {}
record Person(String name, Address address) {}
Object x = new Person("Ada", new Address("Pune"));
if (x instanceof Person(String n, Address(String city))) {
    System.out.println(n + " in " + city);
}
```

7 When to use it - and when not to

CAUTION / TRADE-OFF

They do not copy records or guarantee deep immutability. Patterns can be combined with switch.

8 Interview corner: predict, then explain

QUESTION: Does Point(int x,int y) modify Point?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. It binds local variables to existing component values.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Record patterns: finalized. Release: Java 21. Core idea: Record patterns combine type checking with component extraction, and can be nested to match records inside records.

Watch out: They do not copy records or guarantee deep immutability. Patterns can be combined with switch.

LESSON 32 | JAVA 21 | Sequenced collections

1 The real problem (read this first)

You need first and last elements from ordered collections. Different collection interfaces previously used different APIs or awkward indexing.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
List<String> names = new ArrayList<>(List.of("A", "B"));
String first = names.get(0);
String last = names.get(names.size() - 1);
```

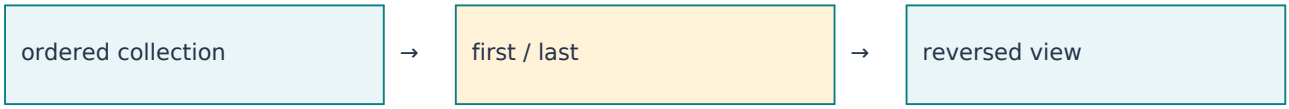
3 What changed and when

Java 21 adds common APIs for collections with a defined encounter order. SequencedMap also offers firstEntry/lastEntry and reversed views.

JAVA 21 FEATURE / MODERN PRACTICE

```
SequencedCollection<String> names =
    new ArrayList<>(List.of("A", "B"));
System.out.println(names.getFirst());
System.out.println(names.getLast());
System.out.println(names.reversed());
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
var names = new ArrayList<>(List.of("A", "B", "C"));
var rev = names.reversed();
names.add("D");
System.out.println(rev); // [D, C, B, A]
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
reversed() often returns a view, not a copy. Empty collections throw NoSuchElementException for getFirst/getLast. HashSet has no defined encounter order.

8 Interview corner: predict, then explain

QUESTION: If you mutate an ArrayList after getting reversed(), can the view change?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: Yes. A reversed view reflects the backing list; it is not a snapshot.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Sequenced collections. Release: Java 21. Core idea: Java 21 adds common APIs for collections with a defined encounter order. SequencedMap also offers firstEntry/lastEntry and reversed views.

Watch out: reversed() often returns a view, not a copy. Empty collections throw NoSuchElementException for getFirst/getLast. HashSet has no defined encounter order.

LESSON 33 | JAVA 21 | Generational ZGC and GC reasoning

1 The real problem (read this first)

An application creates many short-lived objects. You need to understand why the JVM reclaims memory and why GC configuration affects latency.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
for (int i = 0; i < 1000; i++) {
    byte[] temporary = new byte[1024];
}
```

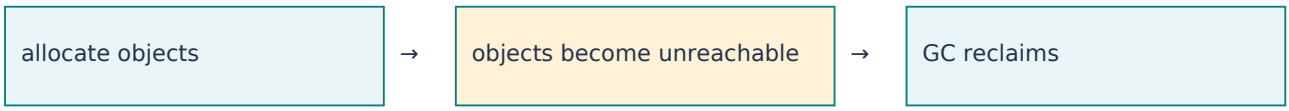
3 What changed and when

Garbage collection (GC) reclaims heap memory occupied by unreachable objects. Generational collectors exploit the observation that many objects die young. ZGC targets low pauses.

JAVA 21 FEATURE / MODERN PRACTICE

```
// Java 21: generational ZGC available.
// Example JVM options:
// java -XX:+UseZGC -XX:+ZGenerational Main
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
Object x = new Object();
x = null; // former object may now be unreachable
// Collection timing is controlled by the JVM.
```

7 When to use it - and when not to

CAUTION / TRADE-OFF

Do not assume GC runs immediately after setting a reference to null. JVM flags vary across releases. Benchmark before tuning.

8 Interview corner: predict, then explain

QUESTION: Does System.gc() guarantee an object is collected immediately?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. It is a request, not a guarantee.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Generational ZGC and GC reasoning. Release: Java 21. Core idea: Garbage collection (GC) reclaims heap memory occupied by unreachable objects. Generational collectors exploit the observation that many objects die young. ZGC targets low pauses.

Watch out: Do not assume GC runs immediately after setting a reference to null. JVM flags vary across releases. Benchmark before tuning.

LESSON 34 | JAVA 21 | Java 21 preview features and migration checklist

1 The real problem (read this first)

You want to use Java 21 confidently without confusing finalized features with previews or breaking an old Java 8 application.

Before looking at the new feature, ask: What behavior does this code need? Who owns the data? What should happen when the input changes?

2 How developers handled it before

OLDER APPROACH / CONTEXT

```
// Java 8 application may rely on old JDK-bundled APIs.
// Some Java EE / CORBA modules were removed in Java 11.
```

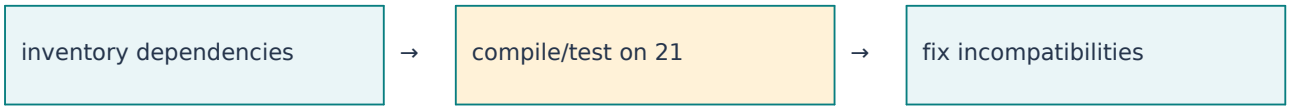
3 What changed and when

A preview is an experimental language/API feature requiring explicit enablement. Migration also means updating dependencies, builds, test suites, and removed APIs.

JAVA 21 FEATURE / MODERN PRACTICE

```
// Java 21 preview examples:
// String templates (preview, later withdrawn)
// Unnamed patterns/variables (preview)
// Structured concurrency and scoped values (preview)
// Foreign Function & Memory API (preview)
// Finalized: virtual threads, record patterns, switch patterns.
```

4 Trace the idea



5 A second working example

TRY THIS AND PREDICT THE OUTPUT

```
// Terminal:
java -version
javac --release 21 Main.java
jdeps --jdk-internals app.jar
```

7 When to use it - and when not to

CAUTION / TRADE-OFF
Do not use withdrawn or preview APIs as if they were permanent Java 21 features. Use supported libraries and run regression tests.

8 Interview corner: predict, then explain

QUESTION: Is a Java 21 preview feature automatically safe to rely on for long-term source compatibility?

Pause. Write your answer and explain which object, branch, or API rule makes it true.

ANSWER: No. Preview designs can change or disappear.

9 Practice ladder

Level 1: Run both versions. Change one input and predict the result before executing.

Level 2: Break the new version on purpose (null, empty input, wrong type, or missing case). Explain the failure.

Level 3: Rewrite one small Java 8 method from your own codebase using this feature. Compare readability and behavior.

10 Remember this

Feature: Java 21 preview features and migration checklist. Release: Java 21. Core idea: A preview is an experimental language/API feature requiring explicit enablement. Migration also means updating dependencies, builds, test suites, and removed APIs.

Watch out: Do not use withdrawn or preview APIs as if they were permanent Java 21 features. Use supported libraries and run regression tests.

MASTER COMPARISON: WHICH LIST SHOULD I USE?

Need	Choice	Reason
Caller should edit membership	new ArrayList<>(source)	Independent, mutable container
Array changes should be visible	Arrays.asList(array)	Fixed-size view backed by array
Read-only live data	Collections.unmodifiableList(source)	Read-only wrapper around original
Read-only snapshot, nulls allowed	unmodifiableList(new ArrayList<>(source))	Separate container + wrapper
Known fixed elements, no nulls	List.of(a,b)	Unmodifiable independent container
Copy existing collection, no nulls	List.copyOf(source) (Java 10)	Unmodifiable snapshot

COMMON INTERVIEW TRAPS: QUICK REVIEW

Question	Correct reasoning
Does final make an ArrayList immutable?	No. final prevents reference reassignment, not object mutation.
Does Arrays.asList allow set?	Yes. It is fixed-size, not unmodifiable.
Does unmodifiableList create a snapshot?	No. It wraps the live backing list.
Does List.of copy mutable elements?	No. It copies references, not the objects.
Does var change Java to dynamic typing?	No. The compiler infers a fixed static type.
Are records deeply immutable?	No. Components may reference mutable objects.
Is Stream.toList mutable?	No. It is unmodifiable.
Are virtual threads faster for CPU-bound work?	Not automatically. They mainly help with waiting-heavy concurrency.
Is Java 21 structured concurrency finalized?	No. It is preview in Java 21.
Does System.gc guarantee collection?	No. GC timing is controlled by the JVM.

FINAL PROJECT: REBUILD A JAVA 8 PAYMENT SERVICE

Start with a mutable payment-method list, a switch-based status label, a DTO, a blocking HTTP call, and a thread pool. Refactor in stages: use defensive snapshots; replace DTO with a record where suitable; use switch expressions; add pattern matching; try virtual threads for I/O tasks; and test behavior at each step. Write down any semantic changes instead of assuming shorter code is equivalent.

Suggested official references

Oracle Java language changes: https://docs.oracle.com/en/java/javase/21/language/java-language-changes-summary.html

OpenJDK JEP index: https://openjdk.org/jeps/0

Java learning portal: https://dev.java/learn/

Oracle Java 21 migration guide: https://docs.oracle.com/en/java/javase/21/migrate/